SAECOIN — BUILD AND RELEASE GUIDE

Document: Build_and_Release.md
Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Status: Planning
Development OS: Windows 11
Available RAM: 16 GB
Purpose: Build, test, package, and release procedures

---

1. PURPOSE

This document defines the standard procedures for building, testing, packaging, and releasing SAECOIN software.

It covers:

- Rust blockchain core builds
- Full node builds
- CLI builds
- Mining software builds
- Wallet builds
- Explorer builds
- Automated CI testing
- Testnet releases
- Mainnet releases
- Version management
- Release security

These instructions are for future development. They do not indicate that the software has already been implemented.

---

2. DEVELOPMENT ENVIRONMENT

2.1 Required Operating System

- Windows 11 64-bit
- Minimum RAM: 8 GB
- Recommended RAM: 16 GB
- SSD storage recommended
- Stable internet connection

The primary development environment is Windows 11 with 16 GB RAM.

2.2 Required Development Tools

The following tools will be required as development progresses:

Tool| Purpose| Priority
Git| Source control| Required
Rust stable| Blockchain development| Required
Cargo| Build and package management| Required
rustfmt| Code formatting| Required
Clippy| Rust linting| Required
Visual Studio Build Tools| Windows native compilation support| As required
Node.js| Frontend development| Later
Android Studio| Android wallet development| Later
JDK| Android development| Later

Tool versions must be selected and documented before implementation.

A "rust-toolchain.toml" file should be added to the repository to control the Rust toolchain version.

---

3. INITIAL BUILD STRATEGY

SAECOIN will use a staged build process.

Stage 1: Rust Core

Build and test:

- Blockchain primitives
- Transaction structures
- UTXO engine
- Block validation
- Consensus engine
- Cryptographic utilities

Stage 2: Node Software

Build:

- Full node
- P2P networking
- Blockchain synchronization
- Database integration
- Node CLI

Stage 3: Mining Software

Build:

- CPU miner
- GPU mining components
- Mining interface
- Block template generation

GPU support may require additional hardware-specific dependencies.

Stage 4: Wallet Software

Build:

- Wallet core
- Windows desktop wallet
- Android wallet

Stage 5: Explorer and API

Build:

- Explorer backend
- REST API
- Explorer frontend
- Blockchain statistics

---

4. RUST WORKSPACE BUILD COMMANDS

Once the Rust workspace has been created, the following commands will be used from the repository root.

4.1 Format Check

cargo fmt --all -- --check

Purpose:

Verify consistent Rust formatting.

4.2 Workspace Check

cargo check --workspace

Purpose:

Check whether the Rust workspace compiles without producing release binaries.

4.3 Unit and Integration Tests

cargo test --workspace

Purpose:

Run all available Rust workspace tests.

4.4 Clippy Analysis

cargo clippy --workspace --all-targets -- -D warnings

Purpose:

Identify common programming mistakes and enforce the project's lint policy.

This strict command should be used after the team adopts and resolves the required lint rules.

4.5 Release Build

cargo build --workspace --release

Purpose:

Build optimized release binaries for the configured Rust workspace.

Important: These commands assume the workspace and package names have been created. They must not be treated as proof that the proposed repository structure already exists.

Only Rust packages should be included as Rust workspace members.

---

5. BUILD CONFIGURATIONS

SAECOIN should maintain separate configurations for:

5.1 Development

Purpose:

- Local development
- Debugging

- Unit testing
- Feature development

Characteristics:

- Debug logging enabled
- Test data permitted
- Development database
- No real-value transactions

5.2 Testnet

Purpose:

- Testing blockchain functionality
- Mining experiments
- Wallet testing
- P2P testing
- Network stability testing

Characteristics:

- Separate network identifier
- Separate genesis block
- Separate data directory
- Test coins only
- Clearly identified testnet wallets

5.3 Mainnet

Purpose:

Production SAECOIN blockchain.

Requirements:

- Finalized consensus rules
- Independently reviewed cryptography
- Security audit
- Verified genesis block
- Stable node implementation
- Network compatibility testing
- Documented recovery procedures

Mainnet must never use development or testnet configuration.

---

6. OUTPUT DIRECTORIES

Build outputs should not be mixed with source files.

Rust build artifacts normally reside in:

target/

Release packages should be stored separately:

releases/
  windows/
  android/
  explorer/
  checksums/
  release-notes/

The "releases/" directory is a proposed packaging location.

Generated build artifacts should not normally be committed to Git.

---

7. VERSION MANAGEMENT

SAECOIN software should use Semantic Versioning.

Format:

MAJOR.MINOR.PATCH

Example:

0.1.0
0.2.0
0.2.1
1.0.0

Meaning:

- MAJOR: Breaking changes
- MINOR: New backward-compatible functionality
- PATCH: Bug fixes and compatible improvements

Before version 1.0.0, interfaces and protocol details may change.

However, once a public network protocol is established, consensus compatibility must be handled through a documented protocol-upgrade process.

Software version changes must not silently change consensus rules.

---

8. GIT AND GITHUB RELEASE WORKFLOW

All source code should be managed through Git.

Recommended branch structure:

main
develop
feature/*
bugfix/*
release/*
hotfix/*

8.1 Development Workflow

1. Create a feature branch.
2. Implement the required functionality.
3. Add or update tests.
4. Run formatting checks.
5. Run compilation checks.
6. Run automated tests.
7. Review the changes.
8. Merge after approval.

8.2 Release Workflow

1. Freeze the release candidate.
2. Run the complete test suite.
3. Verify protocol compatibility.
4. Review security-sensitive changes.
5. Build release binaries.
6. Generate checksums.
7. Prepare release notes.
8. Tag the release.
9. Publish approved release artifacts.

---

9. AUTOMATED CI

GitHub Actions should be used for continuous integration.

A future CI pipeline should perform:

- Rust toolchain setup
- Dependency resolution
- Formatting checks
- Compilation checks
- Unit tests
- Integration tests
- Clippy checks
- Relevant security checks

The CI system should run automatically when:

- Code is pushed
- A pull request is opened
- A release candidate is prepared

9.1 CI Release Gate

A release must not be approved if:

- Compilation fails
- Required tests fail
- Consensus tests fail
- Critical security checks fail
- Required protocol compatibility checks fail

CI passing does not prove that the blockchain is secure.

---

10. RELEASE PACKAGE CONTENTS

A Windows node release should eventually include:

```
SAECOIN-Windows/
    saecoin-node.exe
    saecoin-cli.exe
    saecoin-miner.exe
    README.md
    LICENSE
    CHANGELOG.md
    config-example/
```

Actual executable names may change during implementation.

Wallet applications should be distributed separately.

Private keys, seed phrases, wallet databases, and personal user data must never be included in public release packages.

---

11. CHECKSUMS AND RELEASE SECURITY

Every official release package should include a SHA-256 checksum.

Example command:

```
Get-FileHash .\saecoin-node.exe -Algorithm SHA256
```

Users should be able to verify that downloaded files match the official release.

Release signing should be introduced before public production distribution.

Private signing keys must:

- Never be committed to GitHub
- Never be included in source code
- Never be included in release archives
- Be stored using appropriate secure key-management procedures

Where practical, release builds should be reproducible.

---

12. TESTNET RELEASE POLICY

Before mainnet launch, SAECOIN must operate through a public testnet.

The testnet should evaluate:

- Block production
- CPU mining
- GPU mining
- Transaction validation
- UTXO consistency
- Node synchronization
- Peer discovery
- Network recovery
- Wallet operation
- Explorer accuracy
- Database recovery
- Long-running node stability

Testnet coins must have no promised monetary value.

Testnet and mainnet must use different network identifiers and genesis blocks.

---

13. MAINNET RELEASE REQUIREMENTS

Mainnet release must remain blocked until all critical requirements are satisfied.

Required approvals:

- Consensus specification finalized
- Monetary issuance mathematically verified
- Genesis block independently verified
- Cryptographic implementation reviewed
- Transaction validation tested
- Difficulty adjustment tested
- Chain selection tested
- P2P security tested
- Wallet security reviewed
- Database recovery tested
- Independent security audit completed
- Public testnet completed successfully
- Critical and high-severity issues resolved
- Mainnet launch plan approved

ASIC resistance must be treated as a design objective rather than a guaranteed property.

A six-confirmation payment policy may be recommended for some use cases, but it does not provide absolute finality.

---

14. BACKUP AND RECOVERY

Release and operational procedures must protect:

- Source code
- Release signing keys
- Genesis configuration
- Network configuration
- Build instructions
- Release checksums
- Critical documentation

Node operators should maintain appropriate backups of their node data.

Wallet recovery information must remain under the wallet owner's control.

Public release packages must never contain wallet secrets.

---

15. RELEASE NOTES

Each release should include:

- Version number
- Release date
- Supported operating systems
- New features
- Bug fixes
- Protocol changes
- Known issues
- Upgrade instructions
- Compatibility warnings
- SHA-256 checksums

Any consensus-related change must be explicitly identified.

---

16. RELEASE CATEGORIES

SAECOIN should use these release categories:

Category| Purpose
Alpha| Early internal development
Beta| Wider testing
Testnet| Public network testing
Release Candidate| Final pre-release verification
Stable| Approved production software

A stable software release must not automatically be interpreted as proof that every protocol or security risk has been eliminated.

---

17. FINAL BUILD AND RELEASE PRINCIPLES

1. Never release untested consensus changes.
2. Never mix testnet and mainnet data.
3. Never commit private keys or seed phrases.
4. Never publish binaries without checksums.
5. Never silently change monetary rules.
6. Never skip critical security reviews.

7. Keep build instructions reproducible.
8. Maintain release notes for every version.
9. Preserve compatibility information.
10. Prioritize blockchain correctness over release speed.

DOCUMENT STATUS

Build and Release Guide Version: 1.0
Project: SAECOIN
Status: Planning
Implementation Status: Not Started
Consensus Parameters: Provisional
Mainnet Release: Not Approved